

**OPTIMASI SISTEM HMI SUMBER TERBUKA PADA PLC OMRON BERBASIS
PROTOKOL FINS PADA PT XYZ**



Oleh
DENNY CHRISNANDA
2502124914

FRANS SEBASTIAN
2502121162

GILANG HANSAGITA
2502124403

COMPUTER SCIENCE STUDY PROGRAM
BINUS ONLINE LEARNING
UNIVERSITAS BINA NUSANTARA
JAKARTA

2025

DAFTAR ISI

DAFTAR ISI	i
DAFTAR TABEL	iii
DAFTAR GAMBAR	iv
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Identifikasi Masalah	2
1.3 Rumusan Masalah	2
1.4 Tujuan Penelitian	2
1.5 Manfaat Penelitian	3
1.6 Ruang Lingkup Penelitian	3
1.7 Hipotesis Penelitian	3
1.8 Sistematika Penulisan	4
BAB II TINJAUAN PUSTAKA	5
2.1 Tinjauan Studi	5
2.2 Tinjauan Pustaka	9
2.2.1 Objek Penelitian	9
2.2.2 Perusahaan Manufaktur dan Kebutuhan Otomasi	9
2.2.3 Programmable Logic Controller (PLC)	9
2.2.4 Protokol Komunikasi Industri dan FINS	9
2.2.5 HMI dan Visualisasi Data	10
2.2.6 Teknologi Web Modern: HTML, CSS, JavaScript, TypeScript, Node.js, Angular	10
2.2.7 HMI Open-Source dan Vendor Lock-In	11
2.2.8 Electron: Aplikasi Desktop Berbasis Web	12
2.2.9 Pengujian dan Validasi Sistem HMI	12
2.3 Kerangka Pemikiran	13
BAB III METODOLOGI PENELITIAN	15
3.1 Tahapan Penelitian	15

3.2	Alat dan Bahan	17
3.2.1	Perangkat Keras	17
3.2.2	Perangkat Lunak	17
3.2.3	Library Tambahan	17
3.2.4	Platform dan Arsitektur Aplikasi	17
3.2.5	Topologi Sistem dan Komunikasi FINS	18
3.3	Modifikasi UI (Frontend Angular)	18
3.4	Modifikasi Backend (Node.js Konektor FINS)	19
3.5	Metode Pengujian	23
3.5.1	Black-Box Testing	23
3.5.2	White-Box Testing	24
3.5.3	Metrik Evaluasi	24
3.5.4	Validasi dan Reproducibility	25
3.6	Evaluasi Berbasis Responden	25
DAFTAR PUSTAKA		26

DAFTAR TABEL

2.1	Tabel Tinjauan Studi	6
3.1	Metrik dan Target KPI Evaluasi Sistem HMI dengan Konektor FINS	25

DAFTAR GAMBAR

2.1	Diagram kerangka pemikiran penelitian	13
3.1	Flowchart Tahapan Penelitian	15
3.2	Topologi Sistem SCADA FUXA dengan Protokol FINS	18
3.3	Rencana modifikasi antarmuka pengguna (UI) untuk konfigurasi perangkat FINS	18
3.4	Rencana modifikasi antarmuka pengguna (UI) untuk konfigurasi tag perangkat FINS	19

BAB I

PENDAHULUAN

1.1 Latar Belakang

Programmable logic controller (PLC) merupakan perangkat utama dalam otomasi industri. Fungsi utama dari PLC adalah memproses dan mengendalikan mesin dengan pengetahuan yang terbatas mengenai pemrograman komputer (Koondhar et al., 2023). Di lingkungan PT XYZ, perusahaan manufaktur komponen otomotif multinasional, sebagian besar mesin produksi telah menggunakan PLC merek Omron. Data internal menunjukkan bahwa lebih dari 90% mesin produksi pada perusahaan mengandalkan PLC Omron sebagai pusat kendali. Tingginya tingkat adopsi PLC Omron pada lini produksi menjadikannya objek penelitian yang relevan dalam konteks integrasi sistem kendali dengan teknologi digital modern.

Untuk mendukung interaksi antara operator dan mesin yang dikendalikan oleh PLC, dibutuhkan *human-machine interface* (HMI). HMI berperan menampilkan data proses, memberikan kontrol interaktif, serta mendukung pengawasan visual yang memudahkan manajemen produksi (Mujawar, 2023). Selama ini, banyak industri mengandalkan sistem HMI komersial. Sistem HMI komersial memiliki keunggulan dalam pengendalian, pengawasan, dan analisis untuk mengurangi *downtime*, biaya, serta menjaga standar kualitas (Sawal, 2025). Namun, ketergantungan pada vendor tertentu sering menimbulkan masalah biaya lisensi, keterbatasan fleksibilitas, dan risiko *vendor lock-in* (Gularte, 2025). Sebagai alternatif, pendekatan sumber terbuka (*open source*) menawarkan efisiensi biaya, fleksibilitas, serta kemudahan kustomisasi. Salah satu platform HMI/SCADA *open source* yang berkembang adalah FUXA, yang berbasis web, ringan, serta mendukung akses jarak jauh dan integrasi IoT (Chen, 2025). Karakteristik ini menjadikan FUXA sebagai sistem HMI sumber terbuka yang potensial untuk mendukung kebutuhan otomasi cerdas di industri manufaktur.

Pada PLC Omron, komunikasi data umumnya dilakukan menggunakan protokol *factory interface network service* (FINS), yakni standar komunikasi yang memungkinkan pertukaran data antarperangkat secara cepat dan andal. Protokol ini memiliki peran penting dalam menghubungkan PLC dengan sistem HMI, sehingga data proses dapat ditampilkan secara visual dan operator dapat melakukan pengendalian secara efektif. Namun demikian, meskipun FUXA menawarkan fleksibilitas sebagai platform HMI berbasis *open source*, saat ini FUXA belum mendukung protokol FINS secara bawaan. Keterbatasan ini menjadi kendala utama dalam implementasi integrasi FUXA dengan PLC Omron yang digunakan pada lini produksi.

Untuk mengatasi kendala tersebut, diperlukan upaya pengembangan dengan

menambahkan dukungan protokol FINS pada FUXA melalui modifikasi kode sumbernya. Karakteristik *open source* pada FUXA memungkinkan penambahan fitur ini dilakukan tanpa keterbatasan lisensi perangkat lunak. Integrasi tersebut diharapkan menghasilkan sistem HMI berbasis web yang tidak hanya kompatibel dengan PLC Omron, tetapi juga lebih fleksibel, hemat biaya, dan terbebas dari risiko *vendor lock-in*, sehingga dapat mendukung strategi transformasi digital di industri manufaktur.

Berdasarkan uraian di atas, penelitian ini difokuskan pada optimasi sistem HMI sumber terbuka berbasis web yang terintegrasi dengan PLC Omron melalui protokol FINS. Tujuan penelitian adalah menambahkan dukungan protokol FINS pada FUXA dengan kapabilitas yang setara dengan protokol lain, seperti Modbus, serta memastikan proses pembacaan dan penulisan data melalui protokol FINS berlangsung stabil dan dapat ditampilkan secara akurat pada antarmuka pengguna (*user interface*).

1.2 Identifikasi Masalah

Berdasarkan uraian latar belakang di atas, dapat diidentifikasi beberapa permasalahan yang dihadapi PT XYZ terkait kebutuhan sistem SCADA, yaitu:

1. Sistem HMI berbasis *open source* FUXA belum mendukung protokol komunikasi FINS secara bawaan, sehingga tidak dapat langsung digunakan untuk integrasi dengan PLC Omron.
2. Proses pembacaan dan penulisan data melalui protokol FINS belum bisa dipastikan berlangsung secara stabil pada sistem HMI FUXA.

1.3 Rumusan Masalah

Rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana menambahkan fitur protokol FINS pada sistem HMI FUXA dengan fitur yang setara dengan protokol lain seperti Modbus.
2. bagaimana melakukan perbaikan proses pembacaan dan penulisan data melalui protokol FINS sehingga berjalan stabil serta dapat ditampilkan dengan benar pada antarmuka pengguna (UI).

1.4 Tujuan Penelitian

Tujuan dari penelitian ini adalah untuk:

1. Menambahkan fitur protokol FINS pada sistem HMI FUXA dengan fitur yang setara dengan protokol lain seperti Modbus.
2. Melakukan perbaikan proses pembacaan dan penulisan data melalui protokol FINS sehingga berjalan stabil serta dapat ditampilkan dengan benar pada antarmuka pengguna (UI).

1.5 Manfaat Penelitian

Manfaat dari penelitian ini antara lain:

- Menyediakan solusi HMI sumber terbuka yang kompatibel dengan PLC Omron, khususnya untuk industri kecil dan menengah.
- Memberikan kontribusi nyata dalam pengembangan perangkat lunak sumber terbuka di bidang otomasi industri.

1.6 Ruang Lingkup Penelitian

Penelitian ini memiliki ruang lingkup sebagai berikut:

- Fokus pada implementasi komunikasi FINS melalui protokol UDP/TCP.
- Penggunaan area memori standar PLC Omron seperti DM, CIO, W, dan sejenisnya.
- Pengembangan terbatas pada pembacaan dan penulisan tag serta visualisasi nilai melalui UI FUXA.
- Pengujian dilakukan menggunakan perangkat lunak analisis jaringan seperti Wireshark serta PLC fisik.

1.7 Hipotesis Penelitian

Hipotesis dari penelitian ini adalah:

- H1 (alternatif): Implementasi FINS pada FUXA menghasilkan komunikasi yang stabil dan dapat divisualisasikan pada UI.
- H0 (nol): Implementasi FINS pada FUXA tidak menghasilkan komunikasi yang stabil atau tidak dapat divisualisasikan pada UI.

1.8 Sistematika Penulisan

Adapun sistematika penulisan dalam laporan ini adalah sebagai berikut:

- **Bab 1** – Pendahuluan: berisi latar belakang, rumusan masalah, tujuan dan manfaat, ruang lingkup, hipotesis, dan sistematika penulisan.
- **Bab 2** – Tinjauan Referensi: membahas teori dan referensi terkait, seperti protokol FINS, FUXA, dan komunikasi industri.
- **Bab 3** – Metodologi Penelitian: menjelaskan tahapan dan metode penelitian yang digunakan.
- **Bab 4** – Implementasi dan Pengujian: menyajikan proses integrasi FINS pada FUXA serta hasil pengujian.
- **Bab 5** – Kesimpulan dan Saran: berisi simpulan dari hasil penelitian serta rekomendasi untuk pengembangan lebih lanjut.

BAB II

TINJAUAN PUSTAKA

2.1 Tinjauan Studi

Tinjauan studi berisi rangkuman penelitian-penelitian terdahulu yang dijadikan acuan dalam menyusun penelitian ini. Tinjauan tersebut diharapkan dapat memperluas wawasan serta memperdalam pemahaman terhadap teori-teori yang telah dikembangkan sebelumnya. Adapun daftar penelitian terdahulu yang menjadi rujukan dalam penelitian ini disajikan pada Tabel 2.1.

Tabel 2.1: Tabel Tinjauan Studi

No.	Penulis (Tahun)	Judul Artikel	Domain Aplikasi	Platform / Teknologi	Tujuan / Kontribusi
1	Uddin dkk. (2022)	Design and Implementation of an Open-Source SCADA System for a Community Solar-Powered Reverse Osmosis System	Industri Air Bersih	Node-RED, Grafana, InfluxDB, Debian OS, Protokol Firmata, Arduino Mega 2560, Arsitektur SCADA Sumber Terbuka	Merancang dan mengimplementasikan SCADA sumber terbuka untuk sistem desalinasi berbasis tenaga surya. Kontribusi: solusi murah dan terbuka untuk komunitas kecil seperti pedesaan.
2	Omidi dkk. (2023)	Design and Implementation of Node-Red Based Open-Source SCADA Architecture for a Hybrid Power System	Sistem Pembangkit Listrik Terbarukan Hibrida	Node-RED, Grafana, InfluxDB, MQTT, Firmata, ESP32, Raspberry Pi	Mengembangkan SCADA sumber terbuka untuk sistem pembangkit listrik hibrida. Kontribusi: sistem modular, berdaya rendah, dan biaya rendah.

No.	Penulis (Tahun)	Judul Artikel	Domain Aplikasi	Platform / Teknologi	Tujuan / Kontribusi
3	Almas dkk. (2014)	Open Source SCADA Implementation and PMU Integration for Power System Monitoring and Control Applications	Sistem Tenaga Listrik	SCADA BR, PMU, DNP3, RT-HIL, LabVIEW, Statnett SDK, Pachube	Mengembangkan SCADA BR dan integrasi PMU. Kontribusi: evaluasi sistem tenaga waktu nyata dan batasan polling DNP3.
4	Salazar dkk. (2024)	ICSNet: A Hybrid-Interaction Honeynet for Industrial Control Systems	Keamanan Siber Sistem Kendali Industri	FUXA HMI, Honeyd, ICSNet, Factory I/O, Mininet, Modbus TCP, IEC-104, ENIP, SNMP, Nmap, Nikto, Proxy	Mengembangkan ICSNet sebagai honeynet hibrida realistis dan modular untuk ICS. Kontribusi: integrasi honeypot interaksi rendah dan tinggi.

No.	Penulis (Tahun)	Judul Artikel	Domain Aplikasi	Platform / Teknologi	Tujuan / Kontribusi
5	Hazdi dkk. (2017)	Desain SCADA Berbasis Web untuk Otomasi Rumah	Otomatisasi Rumah	PLC, SCADA berbasis web, HMI, OPC, MySQL, Visual Basic, Sensor Arus, Jaringan Ethernet	Mengembangkan SCADA untuk otomatisasi rumah dan mengujinya dalam parameter waktu. Kontribusi: aplikasi SCADA dalam rumah tangga dan penyediaan data eksperimental.
6	Kasun Thushara (2024)	Getting Start with FUXA - Web Based SCADA Tool	SCADA/HMI Industri	FUXA HMI, reTerminal DM, Modbus, MQTT, Debian OS	Mendiskusikan implementasi FUXA SCADA berbasis web di perangkat edge. Kontribusi: solusi murah dan terbuka untuk sistem HMI/SCADA industri.

2.2 Tinjauan Pustaka

2.2.1 Objek Penelitian

Objek penelitian adalah PLC Omron seri CJ2M yang berkomunikasi melalui protokol Factory Interface Network Service (FINS). PLC ini digunakan secara luas di industri manufaktur di Indonesia, termasuk di PT XYZ, sehingga mendukung pentingnya penelitian ini.

Selain itu, perangkat lunak open-source FUXA dipilih sebagai platform HMI berbasis web karena berbasis Node.js dan Angular, mendukung Modbus, Siemens S7, BACnet, OPC UA, dan dapat diperluas dengan protokol baru seperti FINS.

2.2.2 Perusahaan Manufaktur dan Kebutuhan Otomasi

Industri manufaktur modern sangat bergantung pada sistem otomasi untuk meningkatkan efisiensi produksi, konsistensi kualitas, dan pengendalian biaya. Otomasi industri melibatkan penggunaan perangkat keras seperti sensor, aktuator, dan Programmable Logic Controller (PLC) yang terhubung ke sistem kontrol dan pengawasan seperti HMI/SCADA (Supervisory Control and Data Acquisition) (McKinsey & Company, 2023).

Perusahaan seperti *Special Purpose Machine (SPM) Maker* sering kali merancang dan membangun mesin otomatis untuk kebutuhan spesifik di pabrik. Mesin-mesin ini biasanya menggunakan PLC dari berbagai vendor, termasuk Omron, Siemens, dan Allen-Bradley. Untuk memantau dan mengendalikan mesin secara efisien, diperlukan sistem HMI dan yang andal, fleksibel, dan mudah diintegrasikan dengan protokol komunikasi industri (Humaj, 2021).

2.2.3 Programmable Logic Controller (PLC)

PLC adalah perangkat digital berbasis mikroprosesor yang dirancang untuk mengontrol proses otomatis di lingkungan industri. PLC dapat diprogram untuk menjalankan logika kontrol yang kompleks dan sangat tahan terhadap kondisi ekstrem seperti getaran, suhu tinggi, dan interferensi listrik. PLC berfungsi sebagai otak dari sistem otomasi, mengumpulkan data dari sensor dan mengontrol aktuator berdasarkan logika yang telah diprogram (EMQX, 2023).

2.2.4 Protokol Komunikasi Industri dan FINS

Agar PLC dapat terhubung ke perangkat lain, diperlukan protokol komunikasi industri. Protokol ini memungkinkan transfer data secara waktu nyata antara PLC dan HMI. Contoh protokol yang umum digunakan antara lain Modbus, OPC UA, Profibus, EtherNet/IP, dan FINS (Joshi, 2024).

FINS (Factory Interface Network Service) merupakan protokol yang dikembangkan oleh Omron untuk memungkinkan komunikasi antar perangkat di dalam jaringan otomasi industri (Humaj, 2021). FINS mendukung komunikasi melalui UDP dan TCP. Kelebihan FINS antara lain:

- Kompatibel dengan semua seri PLC Omron.
- Dukungan komunikasi jarak jauh melalui pengalamatan jaringan.
- Struktur data fleksibel, seperti area CIO, DM, WR, HR.

2.2.5 HMI dan Visualisasi Data

HMI (Human-Machine Interface) adalah antarmuka antara manusia dan sistem kontrol. HMI menyediakan representasi visual dari proses industri dan memungkinkan operator untuk memantau kondisi sistem dan melakukan intervensi jika diperlukan. Fitur penting dalam HMI meliputi grafik waktu nyata, pengaturan parameter, tampilan alarm, dan trend historis (Thushara, 2024).

2.2.6 Teknologi Web Modern: HTML, CSS, JavaScript, TypeScript, Node.js, Angular

Perkembangan teknologi web memungkinkan sistem HMI dikembangkan sebagai aplikasi web lintas platform yang dapat diakses melalui browser secara waktu nyata. Teknologi utama yang digunakan antara lain:

- **HTML (HyperText Markup Language):** Bahasa standar untuk menyusun struktur dan elemen-elemen dasar halaman web. Bhanarkar et al. (2023).
- **CSS (Cascading Style Sheets):** Digunakan untuk mendesain tampilan visual halaman web, termasuk warna, tata letak, dan responsivitas antarmuka. Singh (2014).
- **JavaScript:** Bahasa pemrograman inti untuk interaktivitas pada web, memungkinkan manipulasi DOM, penanganan event, dan komunikasi asynchronous melalui AJAX. Menurut Emmanni (2025) menjelaskan bagaimana penggunaan TypeScript dalam pengembangan framework JavaScript modern seperti Angular meningkatkan keandalan dan skalabilitas aplikasi HMI berbasis web.
- **TypeScript:** Merupakan superset dari JavaScript yang dikembangkan oleh Microsoft, menyediakan fitur pengetikan statis dan pemrograman berorientasi objek yang kuat.

TypeScript digunakan secara luas dalam pengembangan Angular karena meningkatkan skalabilitas, keamanan, dan maintainability kode. Menurut Scarsbrook et al. (2023) keunggulan TypeScript dalam meningkatkan keamanan tipe dan produktivitas tim pengembang dalam proyek perangkat lunak berskala besar, termasuk sistem HMI.

- **Node.js:** Platform berbasis JavaScript yang berjalan di sisi server (backend). Node.js mendukung arsitektur non-blocking dan event-driven, sehingga sangat cocok untuk aplikasi HMI yang membutuhkan performa tinggi dan komunikasi data waktu nyata. Menurut Ancona et al. (2018) menjelaskan bahwa Node.js memiliki karakteristik yang mendukung pemantauan sistem secara waktu nyata, menjadikannya kandidat yang sesuai untuk penerapan pada sistem Internet of Things (IoT) dan HMI modern.
- **Angular:** Framework frontend modern yang dikembangkan oleh Google. Angular menggunakan TypeScript sebagai bahasa utamanya dan menyediakan pendekatan pengembangan berbasis komponen, dependency injection, serta routing yang efisien. Angular mempermudah pembuatan antarmuka pengguna (HMI) yang dinamis, modular, dan responsif. Menurut Client IO (2023), yang memanfaatkan teknologi frontend modern termasuk Angular untuk membangun antarmuka HMI interaktif.

Dengan kombinasi teknologi tersebut, sistem HMI berbasis web menjadi lebih fleksibel, ringan, dan dapat diakses lintas perangkat tanpa memerlukan instalasi perangkat lunak tambahan (Thushara, 2024).

2.2.7 HMI Open-Source dan Vendor Lock-In

Salah satu tantangan utama dalam dunia industri adalah ketergantungan terhadap vendor atau *vendor lock-in*. Sistem HMI komersial umumnya bersifat tertutup dan berlisensi mahal, sehingga membatasi fleksibilitas pengguna dalam hal integrasi, migrasi, maupun penyesuaian sistem.

Vendor lock-in menyebabkan pengguna hanya dapat menggunakan layanan, protokol, atau perangkat lunak dari penyedia tertentu, sehingga sulit untuk beralih ke penyedia lain tanpa mengeluarkan biaya besar atau menghadapi gangguan layanan. Dalam konteks cloud computing, fenomena ini bahkan menjadi lebih kritis, karena banyak organisasi bergantung pada layanan seperti DBaaS (Database-as-a-Service). Ketika organisasi membutuhkan fitur baru atau terjadi perubahan harga, ketergantungan ini menjadi hambatan besar untuk berpindah platform (Banthia, 2024).

Untuk mengatasi vendor lock-in, banyak organisasi mulai mengadopsi pendekatan berbasis teknologi terbuka dan strategi multi-cloud. Salah satu solusi yang relevan dalam konteks HMI adalah penggunaan sistem open-source seperti FUXA. Dengan menggunakan HMI open-source, pengguna memperoleh kendali penuh atas kode sumber, arsitektur, dan kemampuan integrasi sistem, sehingga meminimalkan risiko dikunci oleh vendor tertentu.

FUXA adalah contoh HMI berbasis web yang bersifat open-source dan mendukung berbagai protokol industri seperti Modbus, OPC-UA, dan MQTT. Sistem ini dapat dikustomisasi secara penuh dan tidak tergantung pada penyedia perangkat lunak tertentu (Thushara, 2024).

2.2.8 Electron: Aplikasi Desktop Berbasis Web

Electron adalah framework open-source yang memungkinkan pengembangan aplikasi desktop lintas platform (Windows, macOS, dan Linux) menggunakan teknologi web seperti HTML, CSS, dan JavaScript, dikombinasikan dengan Node.js dan Chromium. Framework ini banyak digunakan untuk membangun aplikasi desktop modern karena kemampuannya menjalankan kode frontend dan backend dalam satu kerangka kerja yang terintegrasi (Electron Team, 2024).

Beberapa keunggulan utama Electron antara lain:

- **Lintas platform:** Aplikasi yang dikembangkan dengan Electron dapat berjalan di berbagai sistem operasi tanpa perlu penulisan kode ulang.
- **Teknologi web modern:** Memanfaatkan ekosistem luas dari JavaScript, TypeScript, dan pustaka web.
- **Stabil dan enterprise-grade:** Digunakan oleh perusahaan besar seperti Microsoft (Visual Studio Code), OpenAI (ChatGPT Desktop), Discord, dan Slack.
- **Kemudahan integrasi native:** Mendukung pemanggilan kode native (C++, Rust, dsb) jika diperlukan.

Dengan bundling Chromium dan Node.js, Electron memberikan kendali penuh terhadap tampilan dan fungsionalitas aplikasi desktop, sekaligus menghindari keterbatasan atau bug dari webview bawaan sistem operasi (Electron Team, 2024).

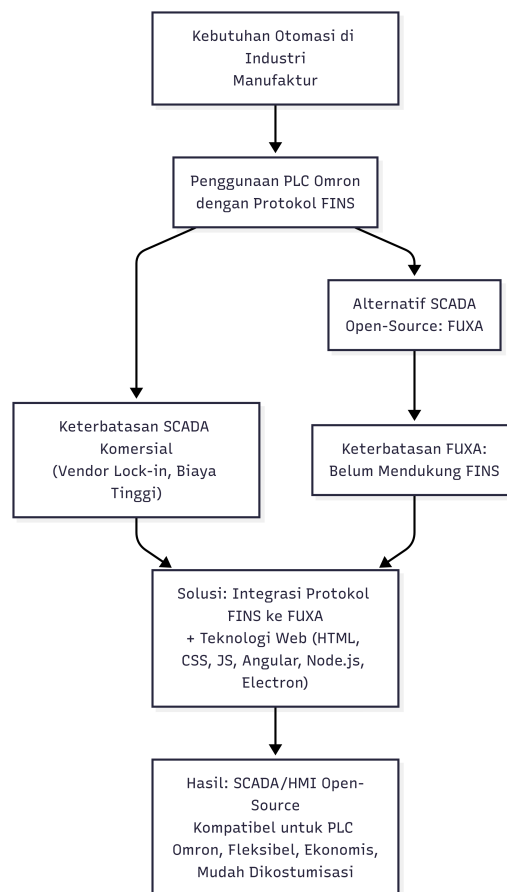
2.2.9 Pengujian dan Validasi Sistem HMI

Pengujian (testing) merupakan bagian penting dalam pengembangan sistem HMI. Pengujian bertujuan untuk memastikan sistem dapat membaca dan menulis data secara benar,

menangani kondisi ekstrem, serta menampilkan visualisasi data yang akurat. Pengujian biasanya mencakup:

- **Unit testing:** Memastikan setiap komponen bekerja sesuai fungsi.
- **Integration testing:** Memastikan komunikasi antara komponen berjalan lancar.
- **System testing:** Menguji sistem secara menyeluruh dalam kondisi nyata atau simulasi.
- **Network traffic analysis:** Menggunakan Wireshark atau alat sejenis untuk memantau paket FINS dalam jaringan (Almas & Vanfretti, 2014).

2.3 Kerangka Pemikiran



Gambar 2.1: Diagram kerangka pemikiran penelitian

Berdasarkan kajian pustaka yang telah dibahas pada bagian 2.2, penelitian ini berangkat dari kebutuhan otomatisasi di industri manufaktur yang semakin tinggi seiring dengan perkembangan teknologi digital dan penerapan konsep Industri 4.0. PT XYZ sebagai perusahaan manufaktur komponen otomotif menggunakan *Programmable Logic Controller*

(PLC) Omron yang berkomunikasi melalui protokol *Factory Interface Network Service (FINS)* untuk mengendalikan sebagian besar mesin produksinya. Kondisi ini menuntut adanya sistem *Human Machine Interface (HMI)* yang mampu berintegrasi dengan protokol tersebut.

Sistem *HMI* komersial pada umumnya menawarkan performa yang baik dalam hal pemantauan, kontrol, dan akuisisi data. Namun, sistem ini seringkali menimbulkan permasalahan *vendor lock-in* yang menyebabkan perusahaan harus bergantung pada penyedia tertentu dengan konsekuensi keterbatasan fleksibilitas integrasi. Untuk mengatasi hal tersebut, solusi berbasis sumber terbuka seperti *FUXA* dipandang lebih sesuai karena memberikan fleksibilitas, efisiensi biaya, serta kemampuan kustomisasi sesuai kebutuhan perusahaan.

Meskipun demikian, *FUXA* belum mendukung protokol *FINS* secara bawaan, sehingga tidak dapat langsung digunakan untuk berkomunikasi dengan *PLC Omron* di PT XYZ. Oleh karena itu, penelitian ini berfokus pada pengembangan integrasi protokol *FINS* ke dalam *FUXA*, dengan memanfaatkan teknologi *web* modern seperti *HTML*, *CSS*, *JavaScript*, *TypeScript*, *Angular*, dan *Node.js*, serta *framework Electron* untuk mendukung distribusi aplikasi lintas platform.

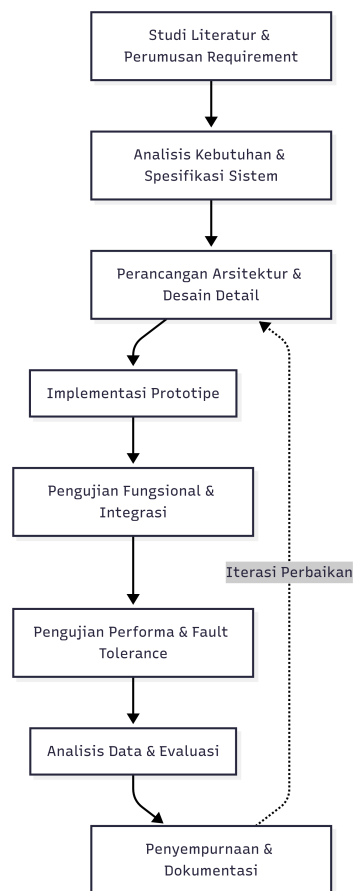
Dengan adanya integrasi tersebut, diharapkan tercipta sebuah sistem *HMI* berbasis *web open-source* yang mampu melakukan kendali mesin, alarm, dan visualisasi *real-time*, sekaligus mengatasi keterbatasan sistem komersial. Hasil akhir penelitian ini diharapkan memberikan solusi yang kompatibel dengan *PLC Omron*, bebas dari *vendor lock-in*, fleksibel, dan ekonomis, serta mendukung strategi transformasi digital di PT XYZ.

BAB III

METODOLOGI PENELITIAN

3.1 Tahapan Penelitian

Tahapan penelitian dijalankan secara iteratif dengan pendekatan *iterative prototyping*, yaitu siklus berulang antara analisis, desain, implementasi, pengujian, dan evaluasi. Diagram alir (flowchart) tahapan penelitian disajikan pada Gambar 3.1.



Gambar 3.1: Flowchart Tahapan Penelitian

Tahapan rinci penelitian adalah sebagai berikut:

1. Studi literatur dan perumusan requirement

Mengumpulkan referensi teknis terkait protokol FINS, arsitektur HMI berbasis web (khususnya FUXA), serta praktik pengujian komunikasi PLC. Hasil: dokumen requirement fungsional dan non-fungsional.

2. Uji coba pustaka omron-fins di Node.js

Membuat skrip sederhana (`Communication test .js`) untuk menguji apakah pustaka

dapat terkoneksi dengan PLC Omron, melakukan operasi baca, dan menulis nilai ke alamat tertentu. Hasil: validasi awal apakah pustaka berfungsi dengan PLC target.

3. Analisis kebutuhan dan spesifikasi sistem

Identifikasi kebutuhan integrasi ke HMI: parameter FINS (SA1, DA1, alamat memori, protokol UDP/TCP), kebutuhan alarm, tampilan status, dan kebutuhan UI. Hasil: spesifikasi teknis dan use-case.

4. Perancangan arsitektur dan desain detail

Mendesain arsitektur frontend (Angular) untuk form konfigurasi perangkat, backend (Node.js) untuk modul FINS, serta jalur komunikasi antara keduanya. Hasil: diagram arsitektur dan desain skema komunikasi.

5. Implementasi prototipe HMI

Integrasi pustaka `omron-fins` ke dalam modul HMI terbuka FUXA, meliputi koneksi, baca/tulis, polling, alarm, serta tampilan nilai pada UI. Hasil: prototipe fungsional HMI dengan dukungan FINS.

6. Pengujian fungsional dan integrasi

Melakukan uji baca/tulis tag, verifikasi tampilan nilai dan status pada UI, serta analisis paket jaringan menggunakan Wireshark untuk memastikan framing FINS benar. Hasil: laporan bug dan perbaikan iteratif.

7. Pengujian performa sederhana

Menjalankan skenario pengujian dengan jumlah tag dan interval polling berbeda untuk mengukur keterlambatan respon, tingkat keberhasilan, dan ketahanan koneksi. Hasil: dataset pengujian (CSV, log) dan metrik evaluasi.

8. Analisis data dan evaluasi

Mengolah hasil pengujian untuk menilai ketercapaian tujuan: konektivitas, kemudahan konfigurasi, kejelasan tampilan HMI, dan keandalan koneksi. Hasil: analisis dan rekomendasi optimasi.

9. Penyempurnaan dan dokumentasi

Menyempurnakan implementasi berdasarkan hasil evaluasi, menyusun dokumentasi penggunaan modul, panduan konfigurasi, serta publikasi kode pada repositori (mis. GitHub). Hasil: rilis final modul dan panduan.

Pada setiap iterasi siklus di atas, dilakukan evaluasi risiko teknis (mis. *timeout*, *connection error*), mitigasi, dan pencatatan perubahan versi. Tahapan ini memastikan bahwa integrasi FINS pada HMI yang dikembangkan tidak hanya berfungsi secara fungsional tetapi juga andal (reliable) dan terdokumentasi.

3.2 Alat dan Bahan

3.2.1 Perangkat Keras

- Laptop (Intel i7, RAM 16GB, SSD 512GB, OS Windows11).
- Industrial Switching Hub Omron W4S1-05B.
- PLC Omron CJ2M CPU34.

3.2.2 Perangkat Lunak

- Node.js v20.x, Angular v17.x, Electron v28.x.
- Wireshark v4.x untuk analisis paket.
- Visual Studio Code, Git.

3.2.3 Library Tambahan

- `node-omron-fins`.
- Angular Material untuk komponen UI konfigurasi.

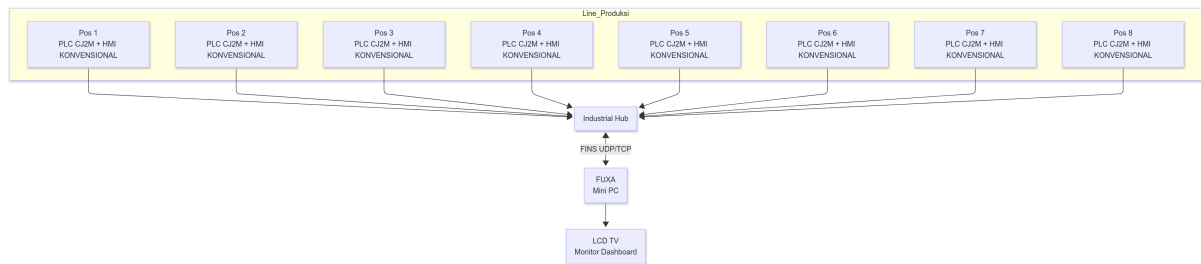
3.2.4 Platform dan Arsitektur Aplikasi

Aplikasi dikembangkan menggunakan framework **Electron**, yang mengemas frontend Angular dan backend Node.js ke dalam executable lintas platform.

Keunggulan pendekatan ini antara lain:

- Distribusi mudah ke Windows dan Linux.
- Performa tinggi dengan mesin V8 (Chromium).
- Akses langsung ke sistem file/logging lokal.
- Portabilitas tinggi di lingkungan industri.

3.2.5 Topologi Sistem dan Komunikasi FINS



Gambar 3.2: Topologi Sistem SCADA FUXA dengan Protokol FINS

Mini PC bertindak sebagai pusat kendali yang berkomunikasi dengan delapan PLC Omron CJ2M melalui protokol FINS berbasis UDP/TCP. Data dari setiap pos kerja diproses dan divisualisasikan secara waktu nyata melalui dashboard utama.

3.3 Modifikasi UI (Frontend Angular)

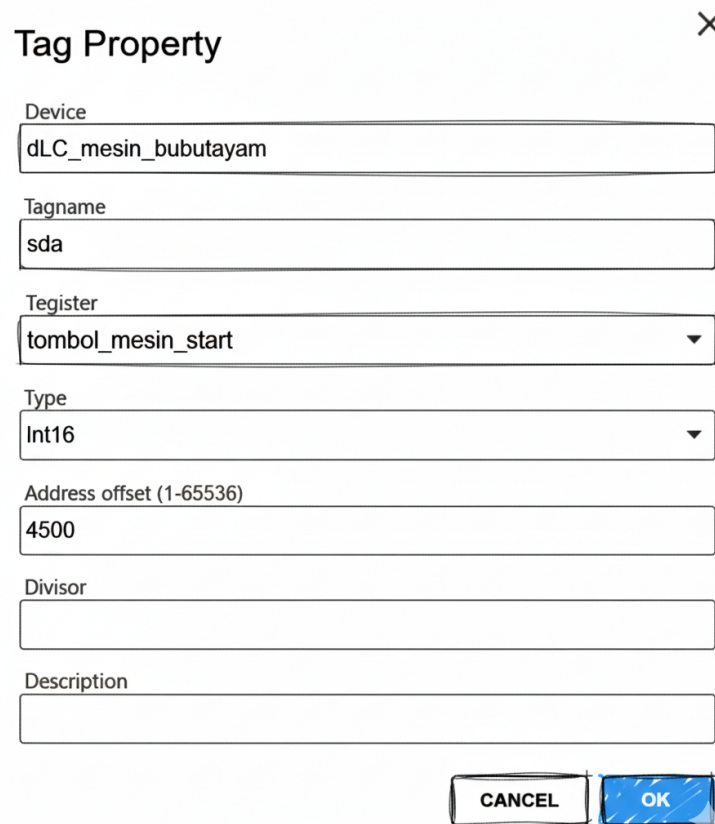
Connections Property			
Name: PLC_Mesin_Cutting			
Type: Fins	Polling: 350 ms	Enable: ☒	
FINS IP Address: 192.168.0.1	SA1 (PC Node): 234	DA1 (PLC Node): 1	Protocol: UDP
[CANCEL] [OK]			

Gambar 3.3: Rencana modifikasi antarmuka pengguna (UI) untuk konfigurasi perangkat FINS

Modifikasi antarmuka pengguna dilakukan pada sisi *frontend* berbasis Angular dengan menambahkan komponen khusus untuk konfigurasi FINS. Perubahan utama antara lain:

- Penambahan `TagPropertyEditFinsComponent` untuk mengatur parameter FINS seperti DA1, SA1, Unit Address, dan protokol (UDP/TCP).
- Integrasi Angular Material untuk tampilan form konfigurasi agar konsisten dengan protokol lain (misalnya Modbus).
- Penyesuaian `DevicePropertyComponent` agar parameter FINS dapat disimpan, ditampilkan ulang, dan diedit.

- Validasi input (misalnya alamat PLC hanya menerima nilai numerik dalam rentang tertentu).



The image shows a 'Tag Property' dialog box with a close button (X) in the top right corner. The dialog contains several input fields and dropdown menus:

- Device:** A text box containing 'dLC_mesin_bubutayam'.
- Tagname:** A text box containing 'sda'.
- Tegister:** A dropdown menu with 'tombol_mesin_start' selected.
- Type:** A dropdown menu with 'Int16' selected.
- Address offset (1-65536):** A text box containing '4500'.
- Divisor:** An empty text box.
- Description:** An empty text box.

At the bottom right, there are two buttons: 'CANCEL' and 'OK'.

Gambar 3.4: Rencana modifikasi antarmuka pengguna (UI) untuk konfigurasi tag perangkat FINS

Selain perencanaan awal (Gambar 3.3), implementasi nyata form edit tag ditunjukkan pada Gambar 3.4. Antarmuka ini memungkinkan pengguna untuk menambahkan atau mengubah konfigurasi tag FINS secara langsung melalui dialog khusus, termasuk:

- Pemilihan area memori (CIO, DM, WR, HR).
- Input alamat numerik sesuai range yang didukung PLC Omron.
- Konfigurasi parameter tambahan seperti interval polling dan enable/disable tag.
- Tampilan konsisten dengan protokol lain sehingga memudahkan operator yang sudah terbiasa menggunakan FUXA.

3.4 Modifikasi Backend (Node.js Konektor FINS)

Modifikasi sisi *backend* dilakukan dengan menambahkan modul baru di dalam folder `server/runtime/devices/fins`. Perubahan meliputi:

- Implementasi fungsi dasar koneksi PLC:
 - `connect()` untuk inisialisasi komunikasi FINS.
 - `read()` untuk membaca nilai alamat PLC.
 - `write()` untuk menulis nilai ke alamat PLC.
 - `poll()` untuk melakukan polling berkala.
- Mekanisme `retry` jika koneksi terputus atau terjadi *timeout*.
- Pengaturan logging error agar memudahkan debugging.
- Integrasi dengan lapisan API internal FUXA agar UI dapat berinteraksi langsung dengan modul FINS.

Contoh pseudocode alur kerja modul konektor FINS ditunjukkan pada Snippet ??.

Pseudocode: DeviceFins Module

```

1. LOAD library "omron-fins"
   IF gagal -> log error, modul tidak tersedia

2. FUNCTION DeviceFins(data, logger, events, runtime):
   INIT:
       client = null
       values = {}
       isConnected = false
       isConnecting = false
       reconnectTimer = null
       deviceId = data.id
       deviceName = data.name
       deviceTags = list dari data.tags
       options = data.property

       host = options.address OR default
       port = options.port OR default
       protocol = options.FinsProtocol OR 'UDP'

```

```
SA1, DA1 = options.SA1, options.DA1
```

LOG konfigurasi device

```
FUNCTION scheduleReconnect():
```

```
  IF sudah connect/connecting -> return
```

```
  CANCEL reconnectTimer jika ada
```

```
  SET reconnectTimer = timeout 3 detik:
```

```
    log "Trying reconnect"
```

```
    CALL connect()
```

```
    IF gagal -> log "Reconnect failed"
```

```
FUNCTION connect():
```

```
  RETURN Promise
```

```
    IF fins library tidak ada -> reject
```

```
    isConnected = true
```

```
    BERSIHKAN client lama jika ada
```

```
    CREATE client baru FinsClient(host, port, SA1, DA1, protocol)
```

```
    client.onError:
```

```
      log error
```

```
      set isConnected=false, isConnecting=false
```

```
      CALL disconnect()
```

```
      CALL scheduleReconnect()
```

```
    client.onTimeout:
```

```
      log timeout
```

```
      set isConnected=false, isConnecting=false
```

```
      CALL disconnect()
```

```
      CALL scheduleReconnect()
```

```
  Jika sukses:
```

```
    set isConnected=true, isConnecting=false
```

```
    emit events "status:connect-ok"
```

```
    resolve()
```

```
FUNCTION disconnect():
```

```

HAPUS semua listener client
PUTUS client jika ada
CLEAR reconnectTimer
set isConnected=false
emit events "status:connect-off"

FUNCTION stop() = disconnect

FUNCTION isConnected() RETURN isConnected

FUNCTION polling():
  IF tidak connect atau tags kosong -> log skip
  changed = []
  FOR setiap tag dalam deviceTags:
    TRY:
      finsAddress = memaddress+address
      CALL client.read(finsAddress)
    ON reply:
      val = response.values[0]
      now = timestamp
      IF val berbeda dari values[tag.id]:
        update values
        tambahkan ke changed
        IF addDaq tersedia:
          simpan data (val, ts)
    ON error:
      log error, set isConnected=false
      CALL scheduleReconnect()
  CATCH err -> log polling exception
  IF changed tidak kosong:
    emit events "device-value:changed"

FUNCTION getValues():
  RETURN semua pasangan (id, value)

```

```

FUNCTION getValue(tagId):
    RETURN {id, value, ts=now}

FUNCTION getStatus():
    RETURN "connect-ok" jika isConnected else "connect-off"

FUNCTION load(newData):
    jika newData.tags ada -> perbarui deviceTags

FUNCTION setValue(tagId, value):
    cari tag sesuai id
    IF client dan tag ada:
        finsAddress = memaddress+address
        client.write(finsAddress, [value])
        ON error: log gagal tulis
        ELSE: log sukses, update values

```

3. MODULE EXPORT:

```

init(): kosong
create(data,...): RETURN new DeviceFins

```

3.5 Metode Pengujian

Pengujian mencakup pengujian fungsional (*black-box*), struktural (*white-box*), serta evaluasi performa.

3.5.1 Black-Box Testing

- **Konfigurasi:** Uji penyimpanan dan pemanggilan parameter (DA1, SA1, Unit Address, protocol).
- **Baca/Tulis Data:** Verifikasi polling nilai PLC dan penulisan tag melalui UI.
- **Alarm:** Validasi pemicu alarm dan notifikasi.

- **UI:** Penggunaan antarmuka tanpa error.

3.5.2 White-Box Testing

- **Unit Testing Konektor:** Fungsi `connect()`, `read()`, `write()`, `poll()`.
- **Event Handling:** Uji pembersihan listener untuk mencegah *memory leak*.
- **Error Handling:** Simulasi kegagalan komunikasi dan mekanisme `retry`.
- **Traffic Analysis:** Validasi paket FINS dengan Wireshark sesuai spesifikasi protokol.
- **Integrasi Client-Server:** Penelusuran alur data dari UI (Angular) ke backend Node.js.

3.5.3 Metrik Evaluasi

Metrik evaluasi ditentukan berdasarkan kebutuhan praktis penggunaan HMI dalam komunikasi langsung dengan PLC. Tabel berikut merangkum metrik dan target kinerja yang diharapkan:

Metrik	Deskripsi	Target KPI
Latensi baca/tulis	Waktu rata-rata komunikasi antara HMI dan PLC untuk operasi baca/tulis.	≤ 100 ms (real-time, tidak terasa delay bagi operator)
Throughput	Jumlah operasi baca/tulis yang dapat diproses per detik.	≥ 100 operasi/detik untuk 100 tag
Persentase keberhasilan komunikasi	Proporsi request yang berhasil dibanding total request.	$\geq 99.5\%$ sukses
Penggunaan CPU/memori	Konsumsi sumber daya oleh aplikasi HMI saat polling aktif.	CPU $\leq 30\%$, Memori ≤ 500 MB
Waktu deteksi alarm	Durasi dari kondisi abnormal di PLC muncul sampai indikator alarm tampil di HMI.	≤ 200 ms
Booting Time	Waktu yang diperlukan aplikasi HMI untuk siap terkoneksi dengan PLC setelah dijalankan.	≤ 10 detik

Tabel 3.1: Metrik dan Target KPI Evaluasi Sistem HMI dengan Konektor FINS

3.5.4 Validasi dan Reproducibility

Untuk memastikan hasil dapat direplikasi:

- Semua kode, konfigurasi, dan skrip pengujian dipublikasikan di GitHub.
- Hasil eksperimen (CSV, log Wireshark) dilampirkan.
- Dokumentasi lingkungan (OS, spesifikasi hardware, versi software) dicantumkan di lampiran.

3.6 Evaluasi Berbasis Responden

Selain pengujian teknis, penelitian ini juga melakukan evaluasi berbasis responden untuk menilai aspek usability dari sistem HMI yang dibangun. Evaluasi dilakukan dengan melibatkan

5–10 responden yang memiliki latar belakang sebagai mahasiswa otomasi industri, teknisi PLC, atau operator produksi.

Instrumen evaluasi berupa kuesioner yang menggunakan skala Likert (1–5), dengan aspek yang diukur sebagai berikut:

1. **Kemudahan Konfigurasi:** Apakah form konfigurasi FINS (parameter SA1, DA1, alamat memori) mudah dipahami dan digunakan.
2. **Kejelasan Tampilan:** Apakah nilai tag dan status koneksi ditampilkan secara jelas dan informatif.
3. **Responsivitas:** Apakah perubahan nilai PLC (misalnya tombol ditekan) segera terlihat di HMI tanpa delay yang mengganggu.
4. **Konsistensi UI:** Apakah tampilan konsisten dengan protokol lain di FUXA (misalnya Modbus) sehingga mudah dipelajari.
5. **Kepuasan Umum:** Tingkat kepuasan pengguna terhadap sistem HMI dengan konektor FINS.

Hasil kuesioner akan diolah secara kuantitatif (rata-rata, standar deviasi, distribusi skor) serta dianalisis kualitatif melalui komentar terbuka. Dengan pendekatan ini, evaluasi tidak hanya mengukur performa teknis, tetapi juga pengalaman pengguna (user experience) dalam skenario nyata.

DAFTAR PUSTAKA

- Almas, M. S., & Vanfretti, L. (2014). "Open source scada implementation and pmu integration". *IEEE PES General Meeting*.
- Ancona, D., Franceschini, L., Delzanno, G., Leotta, M., Ribaud, M., & Ricca, F. (2018). "Towards runtime monitoring of node.js and its application to the internet of things". *arXiv preprint arXiv:1802.01790*. <https://doi.org/10.48550/arXiv.1802.01790>
- Banthia, B. (2024, August 13). *Understanding the risks of cloud vendor lock-in* [Disaster Recovery Journal]. Tessell. https://drj.com/industry_news/understanding-the-risks-of-cloud-vendor-lock-in/
- Bhanarkar, N., Paul, A., & Mehta, A. (2023). "Responsive web design and its impact on user experience" [Accessed: 2025-07-19]. *International Journal of Advanced Research in Science, Communication and Technology*, 50, 50–55. <https://doi.org/10.48175/ijarsct-9259>
- Chen, J. (2025, July). "Top 10 open-source web scada platforms from around the world" [Blog post]. <https://armbasedsolutions.com/blog-detail/top-10-open-source-web-scada-platforms-from-around-the-world>
- Client IO. (2023). "Scada hmi demo using angular and jointjs+" [Accessed: 2025-07-19]. *JointJS Official Demos*. <https://www.jointjs.com/demos/scada>
- Electron Team. (2024). *Why electron* [Accessed: 2025-07-18]. ElectronJS. <https://www.electronjs.org/docs/latest/why-electron>
- Emmani, P. S. (2025). "The role of typescript in enhancing development with modern javascript frameworks". *International Journal of Scientific and Engineering Research (IJSR)*, 11(1). https://www.researchgate.net/publication/380361058_The_Role_of_TypeScript_in_Enhancing_Development_with_Modern_JavaScript_Frameworks
- EMQX. (2023). "Omron fins protocol overview" [Accessed July 2025].
- Gularte, A. (2025, February). "Open vs. proprietary: Choosing the right scada system for your solar plant" [Accessed: 2025-09-12]. <https://blog.norcalcontrols.net/open-vs.-proprietary-choosing-the-right-scada-system-for-your-solar-plant>
- Humaj, P. (2021, April). "Communication with omron fins". <https://d2000.ipesoft.com/blog/communication-omron-fins/>
- Joshi, B. (2024). "A study of the various communication protocols used in plc systems: Modbus, profibus, ethernet/ip and their implementations, evolution and comparative

- analysis”. *International Research Journal of Modernization in Engineering Technology and Science (IRJMETs)*, 6(4). <https://doi.org/10.56726/IRJMETs52882>
- Koondhar, M. A., Kaloi, G. S., Junejo, A. K., Soomro, A. H., Chandio, S., & Ali, M. (2023). “The role of plc in automation, industry, and education: A review”. *Pakistan Journal of Engineering Technology and Science*, 11(2), 22–31. <https://doi.org/10.22555/pjets.v11i2.975>
- McKinsey & Company. (2023). “Is industrial automation headed for a tipping point?” *McKinsey & Company Insights*. <https://www.mckinsey.com/capabilities/operations/our-insights/is-industrial-automation-headed-for-a-tipping-point>
- Mujawar, S. (2023). “Introduction to human–machine interface” [Chapter 1; Accessed via Wiley Online Library]. In *Handbook/book on human–machine interfaces*. Wiley. Retrieved September 26, 2025, from <https://onlinelibrary.wiley.com/doi/abs/10.1002/9781394200344.ch1>
- Sawal, J. (2025, May). “Scada market size, share, trend analysis by 2033” [Report ID: ER_004544]. https://www.emergenresearch.com/industry-report/scada-market?srsId=AfmBOorSdQPJM_IH0PBeriQxoKYbO23L8TR4JyuIII93M6iZJ4ojmwiu
- Scarsbrook, J. D., Utting, M., & Ko, R. K. L. (2023). “Typescript’s evolution: An analysis of feature adoption over time”. *arXiv preprint arXiv:2303.09802*. <https://arxiv.org/abs/2303.09802>
- Singh, V. (2014). “Designing responsive websites using html and css” [Accessed: 2025-07-19]. *International Journal of Scientific & Technology Research*, 3(11), 92–94. <https://www.ijstr.org/final-print/nov2014/Designing-Responsive-Websites-Using-Html-And-Css.pdf>
- Thushara, K. (2024). “Getting start with fuxa - web based scada tool” [Accessed: 2025-07-16]. https://wiki.seeedstudio.com/reTerminal-DM_intro_FUXA/